Microcontrôleur avancé :
DSPic / High Speed ADC + DMA

Tutorial

De Lucas Seguin, HEPIA, lucas.seguin@hes-so.ch

Table des matières

Contexte : 2
Matériel : 2
Matériel (Hardware) : 2
Logiciels (Software) : 3
Schéma/ Montage : 4
Création du projet : 5
Configuration de ADC High-Speed: 7
Ajout et configuration de l'ADC : 8
Configuration de l'UART : 9
Ajout et configuration de l'UART : 9
Configuration du DMA : 11
Finalisation du Code : 13
Visualisation des résultats sur Tera Tern : 14
Contenue du zip : 16
Bibliographies : 16
Annexe : 17

Contexte :

Dans ce tutoriel, nous allons dans un premier configurer l'ADC High-Speed sur la carte dsPIC33CK afin d'effectuer une conversion analogique-numérique ultra-rapide, atteignant une vitesse de 3,25 Ms/s.

Pour tester cette configuration, nous injecterons un signal sinusoïdal en entrée et observerons le résultat de la conversion sur Tera Term via l'UART.

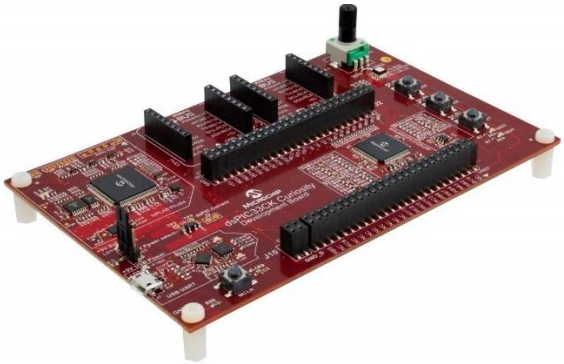
Dans un second temps, nous intégrerons le DMA (Direct Memory Access Controller) afin de stocker les valeurs directement en mémoire, ce qui permettra d'optimiser les performances en réduisant la charge sur le processeur (CPU). Nous afficherons également les valeurs stockées par le DMA en les lisant directement à l'adresse mémoire de stockage.

Matériel :

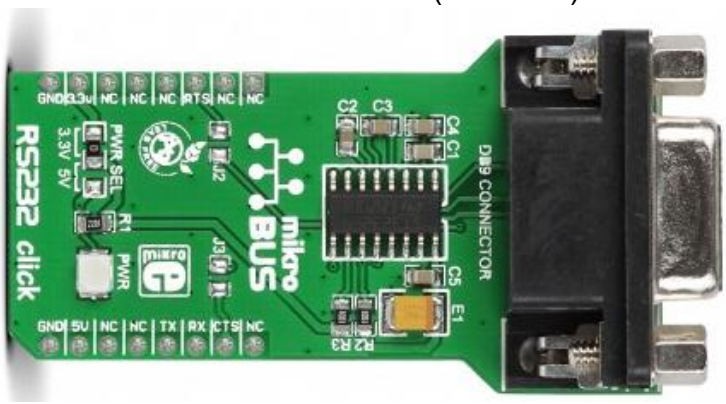
Pour réaliser ce projet, nous avons besoin des éléments suivants :

Matériel (Hardware) :

- Carte dsPIC33CK Curiosity Development Board (Microchip):



- Carte RS232 CLICK mikroBUS (MIKROE) :



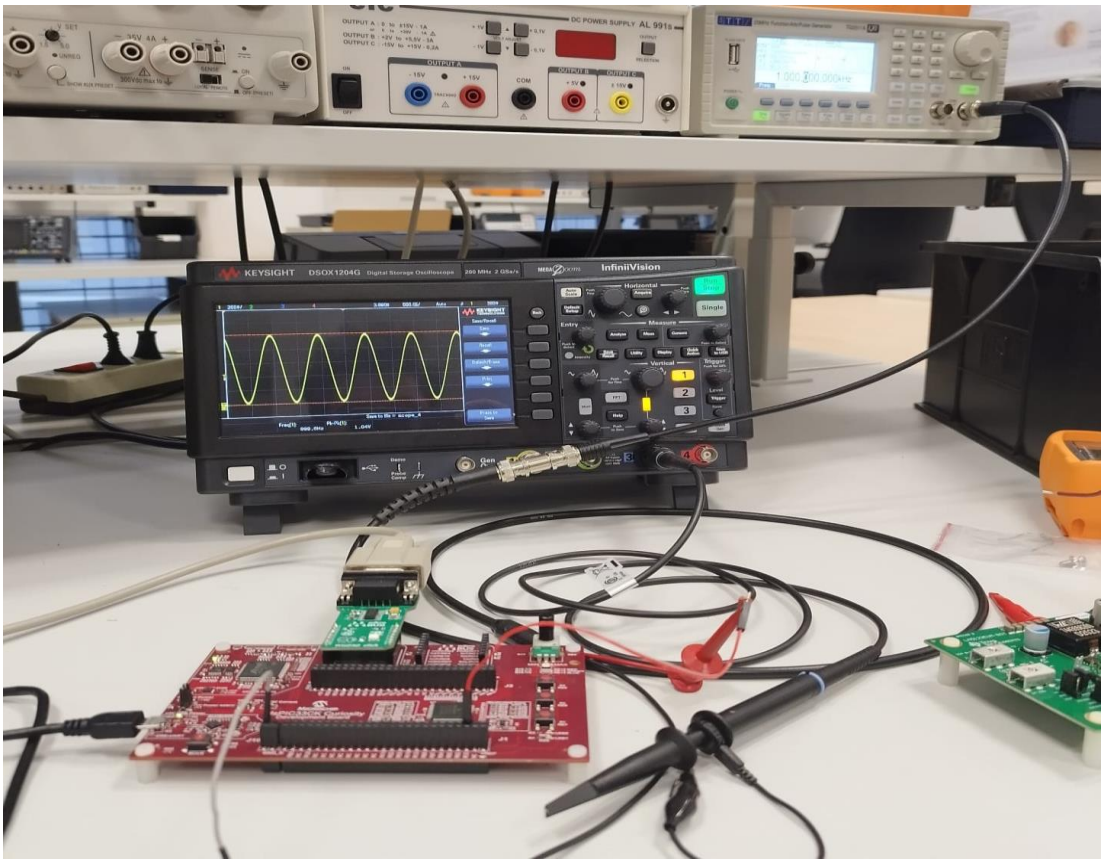
Logiciels (Software) :

- MPLAB X IDE version 6.20 avec l'extension MCC et Application Builder :

- Tera Tern :

Schéma/ Montage :

Voici donc à quoi ressemblerait notre installation pour ce projet :



Nous avons donc besoin pour ce projet :

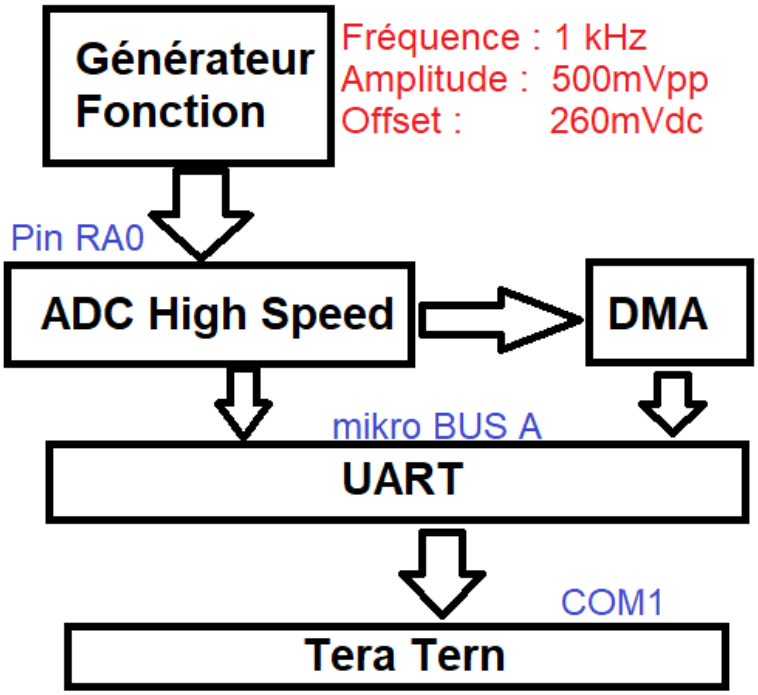
1) Alimentation et programmation :
   - Connecter un câble USB à la carte dsPIC33CK pour l'alimenter et transférer le code via MPLAB X IDE.

2) Connexion du module de communication :
   - Brancher le module RS232 CLICK sur le mikroBUS A de la carte dsPIC33CK.
   - Relier un câble RS232 entre le module et le PC afin d'assurer la communication série.

3) Connexion du signal d'entrée en configurant un générateur de fonctions avec les paramètres suivants :
   - Fréquence : 1 kHz
   - Amplitude : 500 mV
   - Offset : 260 mV
   - Signal Sinusoïdal

   Brancher la sortie du générateur entre la broche GND_D et l'entrée RA0 de la carte dsPIC33CK.

Représentation schématique :

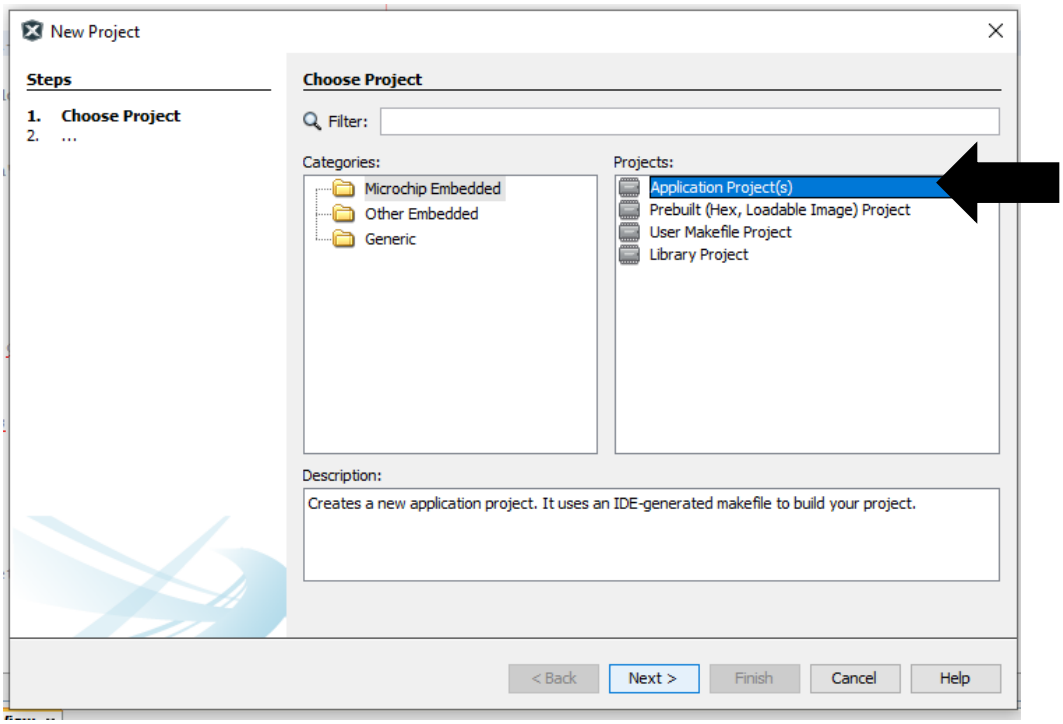


Création du projet :

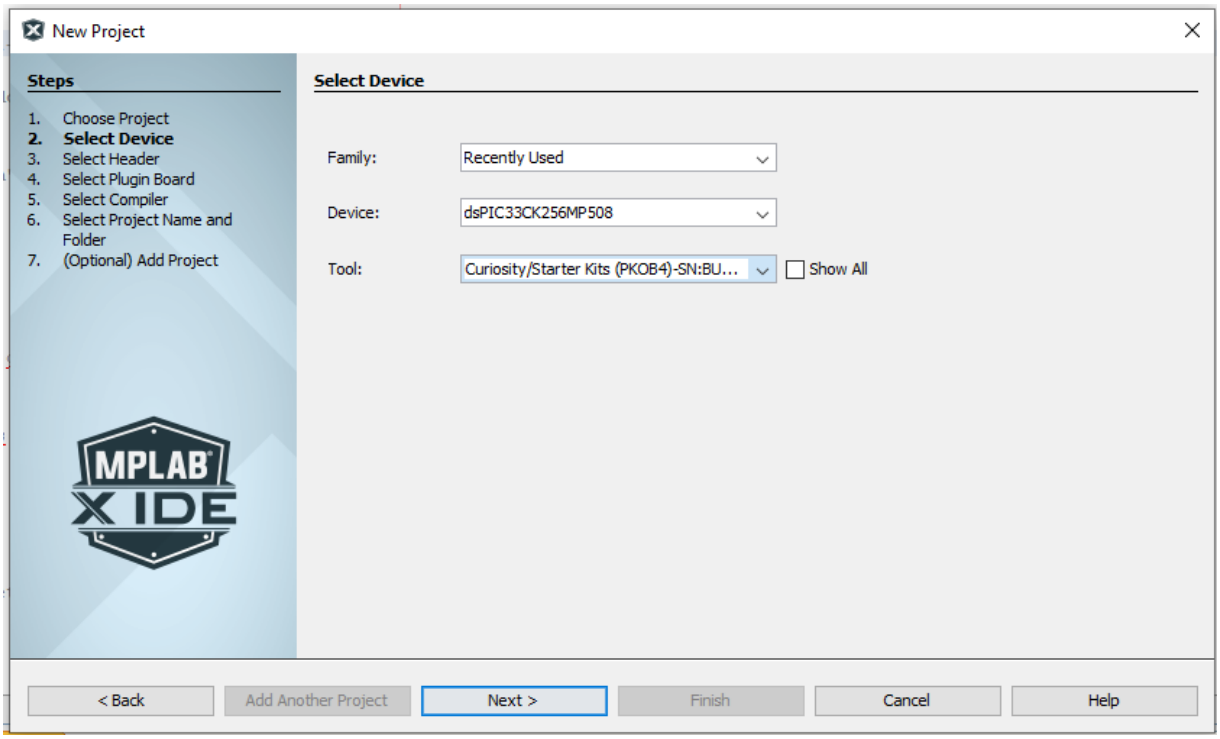
Avant de commencer, nous devons créer un nouveau projet sur MPLAB X IDE.

Étapes à suivre :

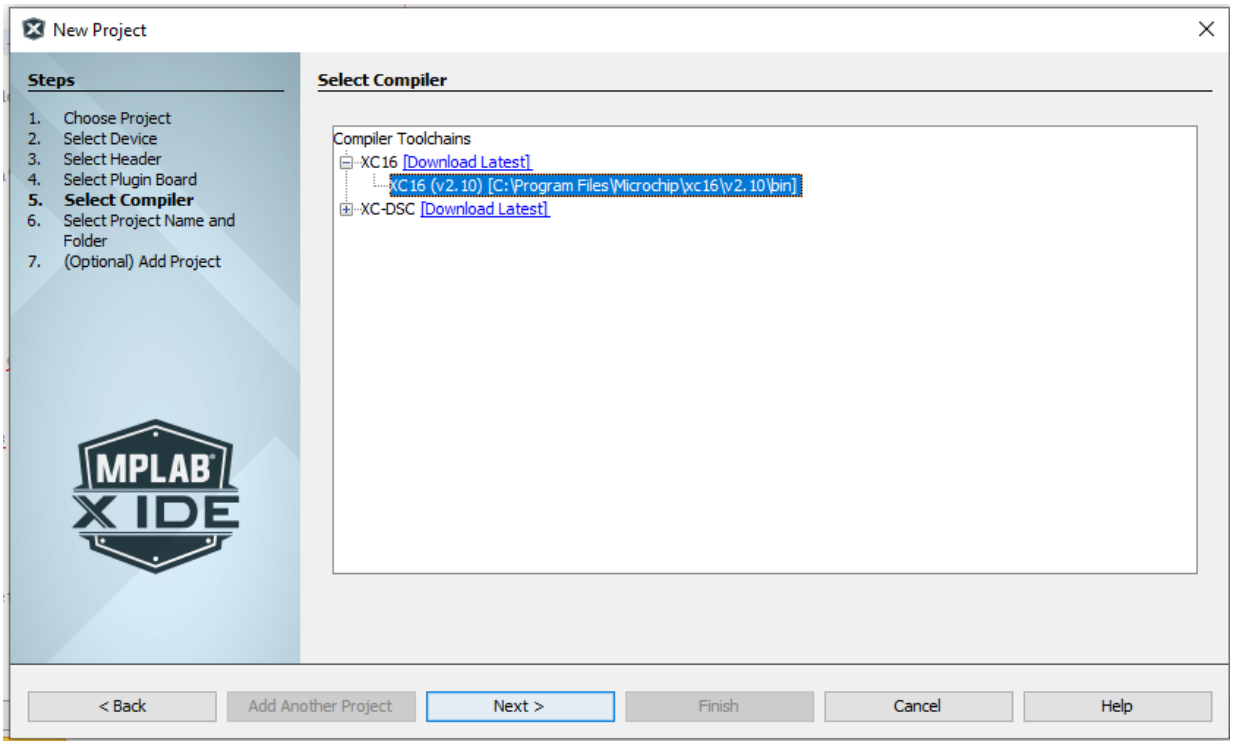
1) Sélection du type de projet
   - Ouvrir MPLAB X IDE et créer un nouveau projet.
   - Choisir le type de projet approprié (Standalone Project).



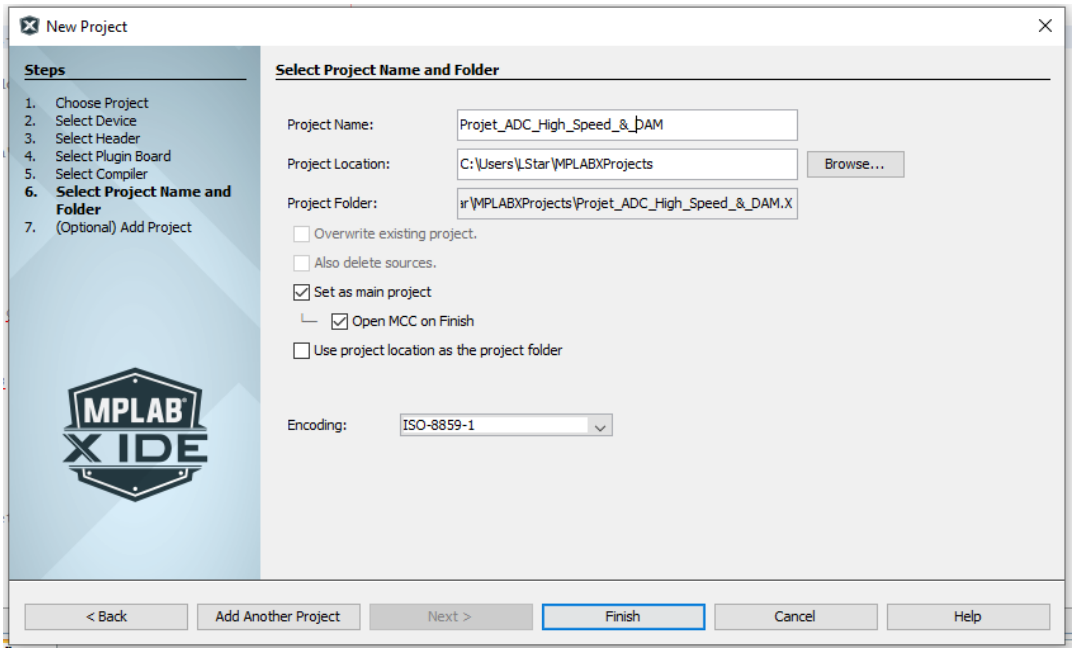
2) Choix du microcontrôleur et du matériel
   - Sélectionner le microcontrôleur utilisé (dsPIC33CK).
   - Choisir le tool de programmation (il s'installe automatiquement lorsque la carte est connectée en USB).



3) Sélection du compilateur
   - Vérifier que le compilateur approprié est sélectionné (ex. XC16 Compiler).



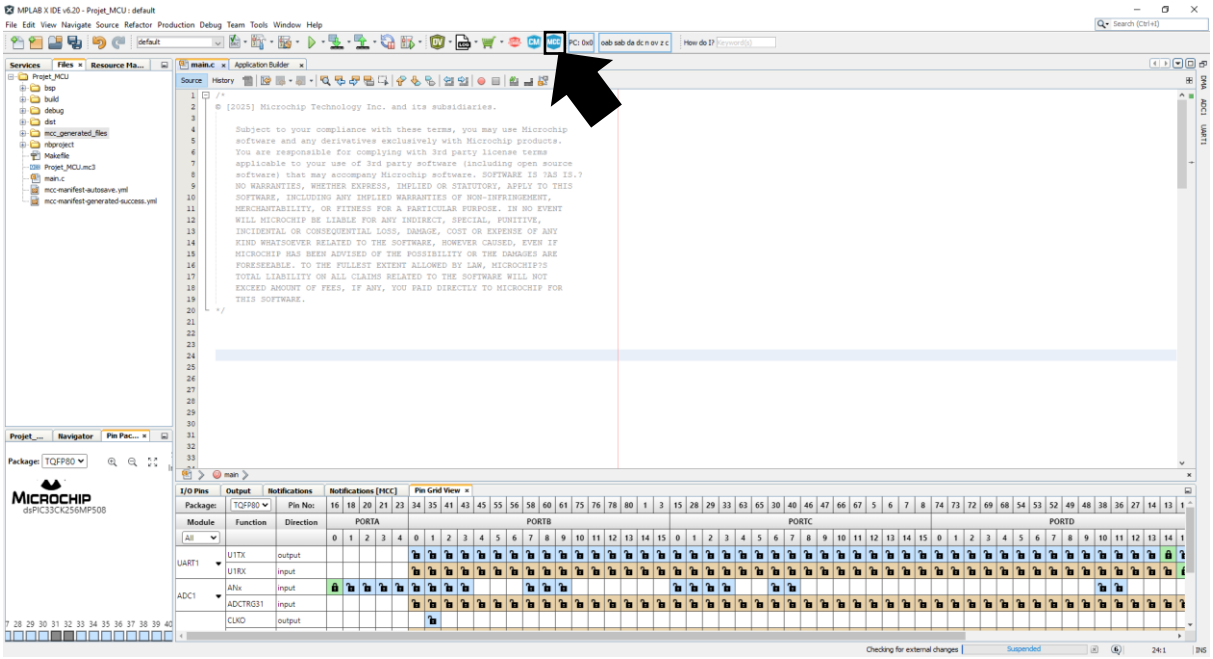
4) Nommer et enregistrer le projet
   - Attribuer un nom au projet et définir son répertoire de sauvegarde.
   - Il est recommandé de laisser l'emplacement par défaut pour éviter tout problème de chemin.



5) Finalisation
   - Une fois toutes les options correctement définies, cliquer sur "Finish" pour générer le projet.

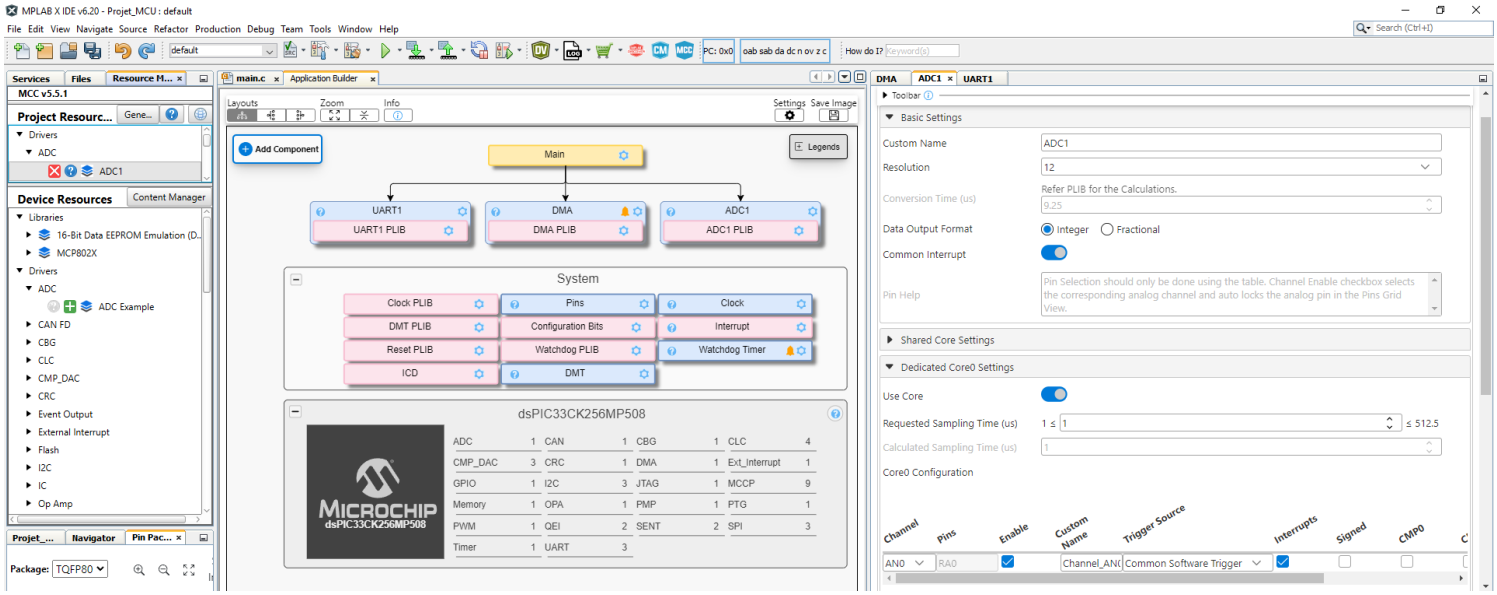
Configuration de ADC High-Speed:

Maintenant que le projet est créé, nous allons configurer l'ADC High-Speed à l'aide de "Application Builder" en lançant MCC (*MPLAB Code Configurator*).



Ajout et configuration de l'ADC :

1) Ajout du module ADC
   - Dans la fenêtre "Device Resources", sous "Divers", ajouter l'ADC au projet.
   - Une nouvelle fenêtre s'ouvre, permettant de configurer les paramètres de l'ADC.



2) Paramétrage des options de base (menu "Basic Settings")
   - **Custom Name :** Nommer le module ADC.
   - **Resolution :** Définir la résolution de conversion analogique-numérique.
   - **Data Output Format** : Déterminer le format des données en sortie.
     - **Integer :** Valeur brute de l'ADC.
     - **Fractional :** Valeur normalisée entre -1 et 1, utile en DSP.
   - **Common Interrupt :** Activer ou désactiver les interruptions communes.

3) Menu "Shared Core Settings" :
   - Ce menu n'est pas utile ici, car il permet de configurer plusieurs ADC classiques sur un même cœur, mais pas un ADC High-Speed.

4) Configuration du cœur dédié (menu "Dedicated Core0 Settings")

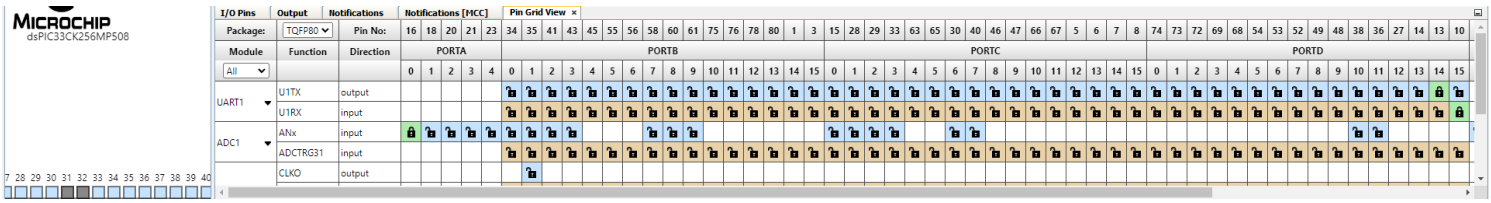
   C'est la partie essentielle, car c'est ici que nous allons paramétrer l'ADC High-Speed.

   - **Use Core :** Activer le cœur dédié.
   - **Channel et Pins** : Sélectionner la broche utilisée pour la conversion
   - **Enable :** Activer la conversion ADC.
   - **Trigger Source :** Choisir le type de déclenchement. Par Exemple :
     - **Software Trigger :** pour déclencher manuellement la conversion.

- **PWM Generator1 ADC Trigger1 :** pour déclencher la conversion baser sur un signal PWM.
- **MCCP9 OC/IC Event :** déclenche la conversion lors d'un événement de capture ou de comparaison produit sur MCCP9.
- **Interrupts :** Activer l'interruption à chaque fin de conversion.
- **Signed / CMP :** Options avancées pour la gestion des seuils et du mode signé.

5) Vérification des broches

- Dans "Pin Grid View", vérifier que la broche AN0 (RA0) est bien configurée en entrée pour l'ADC.



Pour ce projet, nous allons configurer finalement l'ADC High Speed comme ceci :

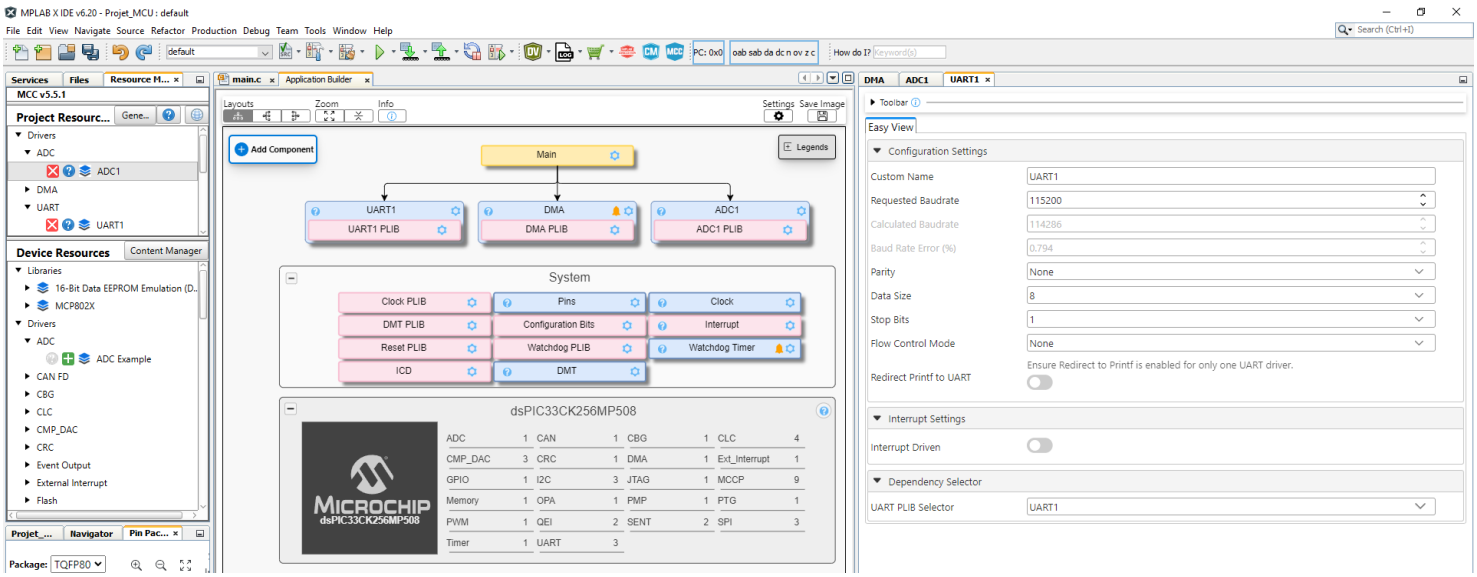
Paramètre	Valeur
Custome Name	ADC1
Resolution	12 bits
Data Output Format	Integer
Common Interrupt	Activé
Use Core	Activé
Channel et Pins	AN0 - RA0
Enable	Activé
Trigger Source	Common Software Trigger
Interrupts	Activé
Signed / CMP	Désactivé

Configuration de l'UART :

L'étape suivante consiste à configurer l'UART en utilisant "Application Builder" dans MCC (MPLAB Code Configurator). Le processus est similaire à celui utilisé pour l'ADC.

Ajout et configuration de l'UART :

1) Ajout du module UART
   - Dans "Device Resources", sous "Divers", ajouter l'UART au projet.
   - Une fenêtre de configuration s'ouvre, permettant d'ajuster les paramètres de l'UART.



2) Configuration des paramètres de base (menu "Easy View")

Ce menu permet de configurer les principaux paramètres de communication série :

- **Custom Name :** Définir le nom du module UART utilisé.
- **Requested Baudrate :** Configurer la vitesse de transmission des données en bits/seconde (doit correspondre à la valeur définie dans Tera Term).
- **Calculated Baudrate & Baud Rate Error (%):**
  - MCC calcule la valeur réelle possible avec les registres internes et affiche l'erreur en %.
  - L'erreur doit être inférieure à ±2 % pour éviter toute perte de données.
- **Parity :** Active ou désactive le contrôle de parité.
- **Data Size :** Définit la taille des données transmises.
- **Stop Bits :** Configure le nombre de bits de stop.
- **Flow Control Mode :** Active ou désactive le contrôle de flux matériel
- **Redirect Printf to UART :** Permet de rediriger la fonction printf() vers le port UART.

3) Gestion des interruptions (menu "Interrupt Settings")
   - **Interrupt Driven :** Permet d'activer les interruptions pour que l'UART génère une interruption lorsqu'un caractère est reçu ou prêt à être transmis.
   - Utile pour éviter d'utiliser les fonctions UART1_IsTxReady() ou UART1_Write() en mode polling.

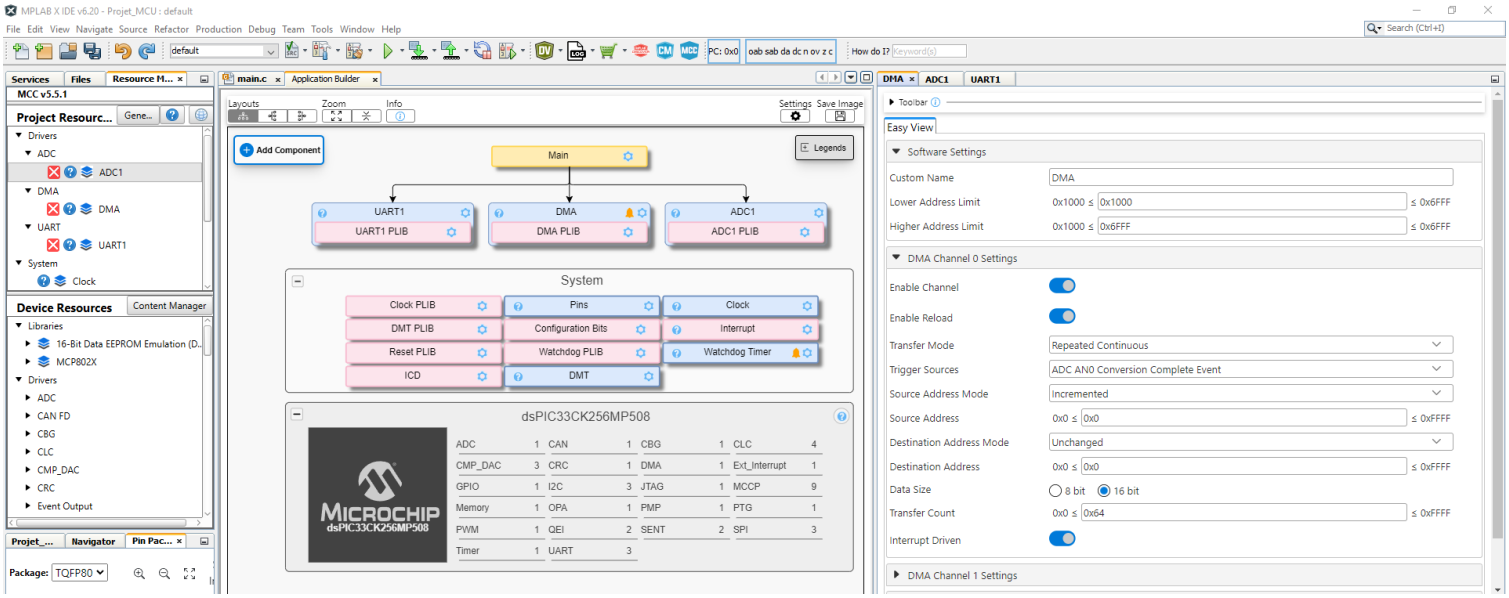
4) Sélection des dépendances (menu "Dependency Selector")
   - Configure automatiquement le UART PLIB Selector pour lier l'UART à la bonne Peripheral Library (PLIB).

La configuration de l'UART retenue pour ce projet est la suivante :

Paramètre	Valeur
Custome Name	ADC1
Requested Baudrate	115200
Parity	None
Data Size	8 bits
Stop Bits	1 bit
Flow Contrôle Mode	None
Redirect Printf to Uart	Desactivé
Interrupt Driven	Désactivé
UART PLIB Selector	UART1

Configuration du DMA :

La dernière étape de la configuration dans MCC concerne le DMA. L'ajout du module se fait de la même manière que pour les précédents périphériques. Une fois ajouté, la fenêtre suivante s'ouvre :



1) Paramètres généraux (Software Settings)

Dans ce menu, nous pouvons configurer :

- **Custom Name :** Permet de personnaliser le nom du module DMA.
- **Lower Address Limit :** Définit la limite inférieure des adresses RAM autorisées pour le DMA. Cette option empêche l'accès à des zones mémoire critiques.
- **Higher Address Limit :** Définit la limite supérieure des adresses accessibles par le DMA.

2) Paramètres du canal DMA (DMA Channel 0 Settings)
   Dans ce menu, nous pouvons configurer :
   - **Enable Channel :** Active ou désactive le DMA.
   - **Enable Reload :** Permet au DMA de redémarrer automatiquement un transfert après la fin d'un cycle, sans nécessiter une reconfiguration manuelle.
   - **Transfer Mode :** Définit le mode de transfert des données entre la source et la destination. Les options disponibles sont :
     - One-Shot : Un seul transfert complet après un seul déclenchement, puis arrêt.
     - Repeated One-Shot : Un transfert par déclenchement, mais avec répétition automatique.
     - Continuous : Le DMA fonctionne en boucle continue sans intervention externe.
     - Repeated Continuous : Utilisé notamment pour l'ADC, permet de remplir un buffer cyclique après chaque déclenchement.
   - **Trigger Sources :** Définit l'événement déclencheur du transfert DMA. Quelques exemples :
     - *ADC AN0 Conversion Complete Event :* Déclenchement après chaque conversion ADC.
     - *UART1 TX/RX Complete :* Activation lorsque l'UART1 transmet ou reçoit un caractère.
     - *Timer Interrupt Event :* Déclenchement après un débordement ou un match d'un Timer.
     - *SPI TX/RX Complete :* Activation après une transaction SPI.
   - **Source Address Mode :** Détermine l'évolution de l'adresse source lors du transfert :
     - *Unchanged* : L'adresse de source reste fixe.
     - *Incremented* : L'adresse source est incrémentée après chaque transfert.
     - *Decremented* : L'adresse source est décrémentée après chaque transfert.
   - **Source Address :** Définit l'adresse mémoire source des données à transférer.
   - **Destination Address Mode :** Détermine l'évolution de l'adresse de destination, avec les mêmes options que Source Address Mode.
   - **Destination Address :** Définit l'adresse de destination des données transférées.
   - **Data Size :** Spécifie la taille des unités de données à transférer (8, 16 ou 32 bits).
   - **Transfer Count :** Détermine le nombre d'éléments transférés par cycle DMA.
   - **Interrupt Driven :** Active ou désactive une interruption DMA à la fin du transfert.

Pour ce projet, nous allons configurer le DMA de la manière suivante :

Paramètre	Valeur
Custom Name	DMA
Lower Address Limit	0x1000
Higher Address Limit	0x6FFF
Enable Channel	Activé
Enable Reload	Activé
Transfert Mode	Repeated Continous
Trigger Sources	ADC AN0 Conversion Complete Event
Source Address mode	Incremented
Source Address	0x0
Destination Address Mode	Unchanged
Destination Address	0x0
Data Size	16 bits
Transfer Count	0x64
Interrupt Driven	Activé

Finalisation du Code :

Dans le fichier main.c, il est essentiel d'inclure correctement tous les fichiers .h générés par MCC afin d'accéder aux configurations précédemment définies :

```
#include <libpic30.h>
#include <xc.h>
#include "mcc_generated_files/system/system.h"
#include "mcc_generated_files/uart/uart1.h"
#include "mcc_generated_files/adc/adc1.h"
#include "mcc_generated_files/dma/dma.h"

#include <stdio.h>
```

Nous allons également ajouter une fonction d'envoi pour l'UART, permettant d'ajuster le délai entre chaque fin de conversion. Cela améliorera la lisibilité des données affichées sur Tera Term. Toutefois, si l'on souhaite observer la vitesse réelle d'acquisition, il suffira de commenter la ligne : *__delay_ms(1)* :

```
void UART_Envoie(const char *str)
{
    while (*str != '\0')
    {
        while (!UART1_IsTxReady()); // Attendre que l'UART soit prêt
        UART1_Write(*str);
        str++;
        __delay_ms(1);  // Pause pour éviter des lectures trop rapides
    }
}
```

Avant d'entrer dans la boucle while(1), nous devons finaliser la configuration de certains paramètres DMA, qui ne sont pas tous accessibles via MCC :

```
67      // Desactive le DMA pour le configurer
68      DMA_ChannelDisable(DMA_CHANNEL_0);
69      // Configurer l'adresse source du DMA (registre ADC1BUF0)
70      DMA_SourceAddressSet(DMA_CHANNEL_0, (uint16_t)&ADCBUF0);
71      // Configurer la destination (notre buffer)
72      DMA_DestinationAddressSet(DMA_CHANNEL_0, adc_buffer);
73      //DMA_DestinationAddressSet(DMA_CHANNEL_0,(uint16_t)&U1TXREG);
74      // Nombre d'échantillons
75      DMA_TransferCountSet(DMA_CHANNEL_0, ADC_BUFFER_SIZE);
76      // Activer DMA Channel 0
77      DMA_ChannelEnable(DMA_CHANNEL_0);
```

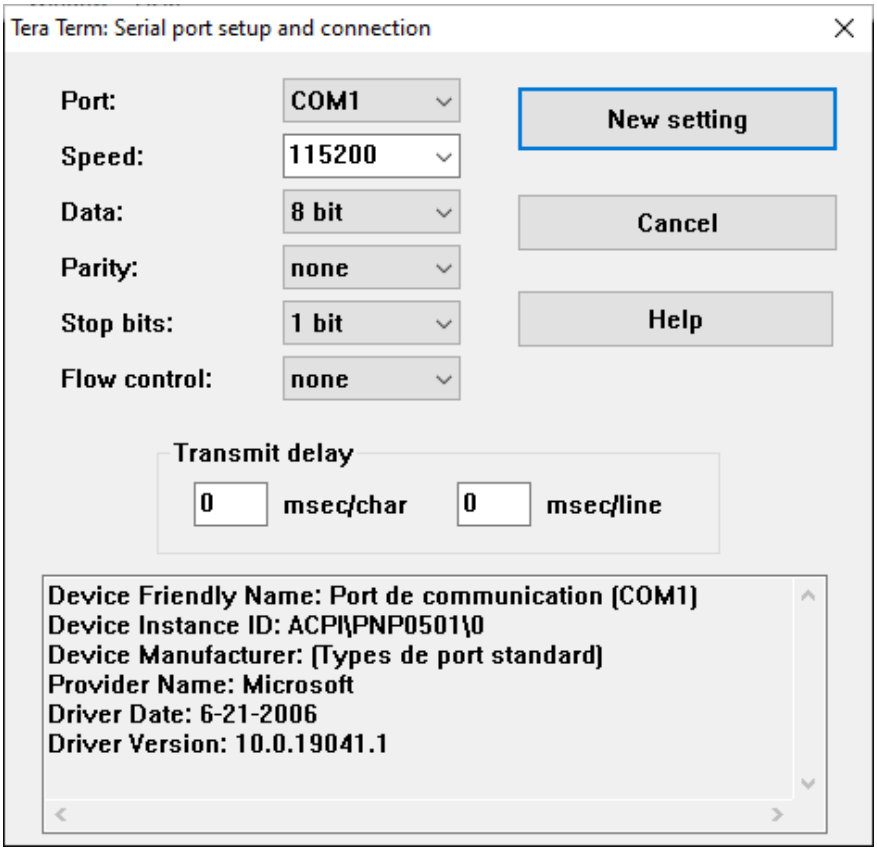
Enfin, il ne faut pas oublier d'initier la conversion ADC manuellement, de récupérer la valeur convertie, de la convertir en chaîne de texte et de l'envoyer via UART :

```
83      while(1)
84      {
85          // Lancer la conversion manuelle via trigger
86          ADC1_SoftwareTriggerEnable();
87
88          // Attendre que la conversion soit terminée
89          while(ADC1_IsConversionComplete(Channel_AN0));
90
91          // Lire le résultat
92          adcValue = ADC1_ConversionResultGet(Channel_AN0);
93
94          // envoie UART
95          sprintf(buffer, "ADC High Speed: %u\r\n", adcValue);
96          UART_Envoie(buffer);
97
98      }
```

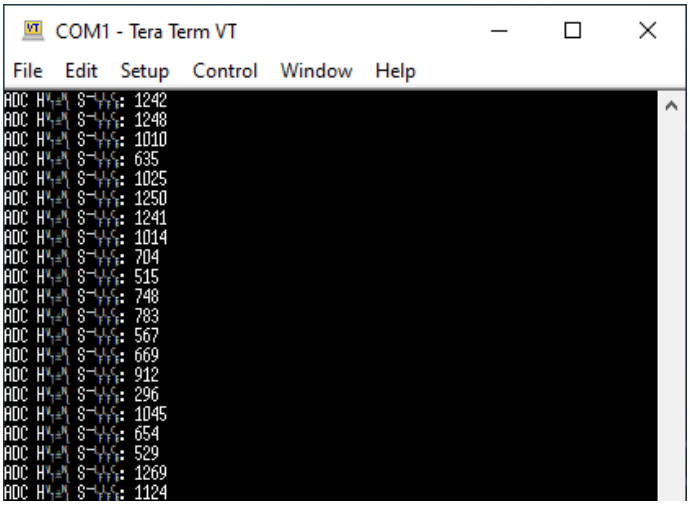
Le code au complet est disponible en **Annexe** de ce document.

Visualisation des résultats sur Tera Tern :

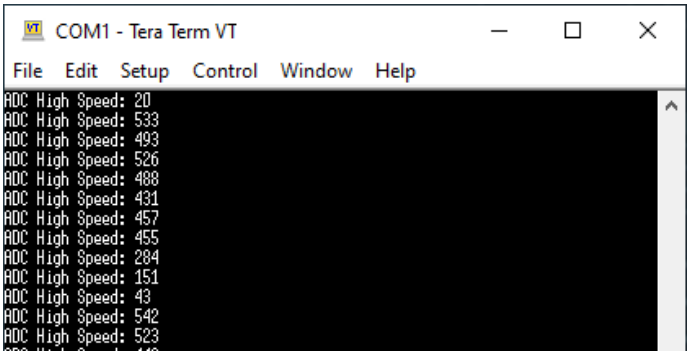
Lors de l'ouverture de Tera Term, il est crucial de sélectionner le bon port de communication et de le configurer de manière identique à l'UART du microcontrôleur. La configuration devra donc être la suivante :



En lançant le code, nous obtenons l'affichage suivant sur Tera Term :



Cependant, nous observons des problèmes d'affichage du texte, qui semblent être dus à l'ajout du DMA, dont l'implémentation n'est pas encore totalement finalisée. Lorsque nous désactivons le DMA, l'affichage est plus propre, mais il ne concerne que la conversion ADC High Speed.



Pour corriger ces erreurs d'affichage et pour poursuivre le projet, nous devons :

- Vérifier la configuration du DMA et s'assurer qu'il fonctionne correctement.
- Utiliser une interruption DMA pour lire la valeur stockée à l'adresse mémoire dédiée et l'afficher sur Tera Term.
- Valider que les données transmises correspondent bien aux valeurs converties par l'ADC High Speed.

Une fois ces corrections effectuées, nous pourrons valider pleinement l'intégration du DMA dans notre projet.

Contenue du zip :

Dans le fichier ZIP, vous trouverez les éléments suivants :

- **Dossier "Datasheet_Document" :** Contient toutes les datasheets et documents utiles pour la réalisation ou la poursuite de ce projet.
- **Dossier "Projet_MCU.X" :** Contient le projet présenté dans ce tutoriel, avec l'ensemble des fichiers source et headers.
- **Dossier "Projet_MCU_Exemple.X" :** Contient des codes d'exemple et de référence fournis par Microchip.
- **Dossier "Projet_MCU_Simple_ADC.X" :** Contient un projet simplifié intégrant uniquement l'ADC High Speed et l'UART.
- **Dossier "Résultat" :** Contient un screen du résultat obtenu sur Tera Tern avec seulement l'ADC High Speed et un autre avec le DMA

Bibliographies :

1) Microchip Technology Inc.
Datasheet du microcontrôleur dsPIC33CK256MP508. Disponible en ligne :
https://www.microchip.com/en-us/product/dspic33ck128mp508

2) Microchip Technology Inc.
Guide d'utilisation de la carte dsPIC33CK Curiosity Development Board. Disponible en ligne :
https://www.microchip.com/en-us/development-tool/dm330030

3) Microchip Technology Inc.
Documentation ADC High Speed (12-bit, Multiple SARs). Disponible en ligne :
https://www.microchip.com/en-us/product/dsPIC33CK256MP508

4) Microchip Technology Inc.
Documentation du contrôleur DMA (Direct Memory Access Controller). Disponible en ligne :
https://www.microchip.com/en-us/product/dsPIC33CK256MP508

5) Microchip GitHub Repository
Exemple d'implémentation du DMA avec ADC vers UART sur dsPIC33C. Disponible en ligne :
https://github.com/microchip-pic-avr-examples/dspic33c-dma-adc-to-uart

Annexe :

Code complete:

```
#define FCY 100000000UL  // Adapter selon l'horloge système


// Déclaration de fichier header et MCC

#include <libpic30.h>

#include <xc.h>

#include "mcc_generated_files/system/system.h"

#include "mcc_generated_files/uart/uart1.h"

#include "mcc_generated_files/adc/adc1.h"

#include "mcc_generated_files/dma/dma.h"

#include <stdio.h>


// Taille du buffer DMA

#define ADC_BUFFER_SIZE 64

volatile uint16_t adc_buffer[ADC_BUFFER_SIZE];  // Unique et correcte définition


// Déclaration de fonction

void UART_Envoie(const char *str);

/*void DMA_Channel0_CallBack(void)
```

```
{
    char msg[40];
    for(uint16_t i = 0; i < ADC_BUFFER_SIZE; i++)
    {
        sprintf(msg, "DMA[%u] = %u\r\n", i, adc_buffer[i]);
        UART_Envoie(msg);
    }
    UART_Envoie("=== End of DMA Buffer ===\r\n");
}*/
void UART_Envoie(const char *str)
{
   // on envoie temps que le caractère retour a la ligne n'est pas detecter
   while (*str != '\0')
   {
      while (!UART1_IsTxReady()); // Attendre que l'UART soit prêt
      UART1_Write(*str); // envoie du caractère
      str++; // caractère suivant
      __delay_ms(1);  // Pause pour éviter des lectures trop rapides sur Tera Tern
   }
}

int main(void)
{
    // Initialisation du système (ADC, UART, etc.)
    SYSTEM_Initialize();

    // Desactive le DMA pour le configurer
    DMA_ChannelDisable(DMA_CHANNEL_0);
    // Configurer l'adresse source du DMA (registre ADC1BUF0)
    DMA_SourceAddressSet(DMA_CHANNEL_0, (uint16_t)&ADCBUF0);
```

```
    // Configurer la destination (notre buffer)
    DMA_DestinationAddressSet(DMA_CHANNEL_0, adc_buffer);
    //DMA_DestinationAddressSet(DMA_CHANNEL_0,(uint16_t)&U1TXREG);
    // Nombre d'échantillons
    DMA_TransferCountSet(DMA_CHANNEL_0, ADC_BUFFER_SIZE);
    // Activer DMA Channel 0
    DMA_ChannelEnable(DMA_CHANNEL_0);

    uint16_t adcValue;
    char buffer[30];

    while(1)
    {
        // Lancer la conversion manuellement via trigger
        ADC1_SoftwareTriggerEnable();

        // Attendre que la conversion soit terminée
        while(ADC1_IsConversionComplete(Channel_AN0));

        // Lire le résultat
        adcValue = ADC1_ConversionResultGet(Channel_AN0);

        // envoie UART
        sprintf(buffer, "ADC High Speed: %u\r\n", adcValue);
        UART_Envoie(buffer);

    }
}
```